

Building Reliable Autonomous Agents with the Claude Agent SDK

The Core Architectural Insight

The Claude Agent SDK doesn't require you to build your own agent loop—Claude Code already handles tool sequencing, error recovery, context management, and retry decisions internally. Your job shifts from *building the loop* to *describing the workflow*. The SDK exposes this runtime programmatically via `query()` and `ClaudeSDKClient`, giving you the same agentic loop that powers Claude Code, accessible from Python.[1][2]

This matters for your Daily Driver setup because it means most "reliability" work is about **configuring the harness well** and **handling the edges the SDK doesn't cover**, not reimplementing retry logic from scratch.

Error Recovery and Retry Logic

What the SDK Handles Natively

The Claude Agent SDK's internal loop already manages tool sequencing failures, re-reads files when edits don't apply cleanly, and decides when to retry tool calls. When a tool call fails, Claude sees the error in its context and adapts—this is model-level error recovery, not code-level retry logic.[2]

The SDK also provides a `fallback_model` option: if the primary model fails (rate limit, outage, capacity issue), it automatically falls back to the specified model.[1]

```

options = ClaudeAgentOptions(
    model="claude-sonnet-4-20250514",
    fallback_model="claude-haiku-4-5-20251001", # Auto-fallback on primary model
    max_turns=30,
    max_budget_usd=0.50,
)

```

What You Need to Build Yourself

The SDK doesn't provide automatic retry of the *entire agent run* if the subprocess crashes or the CLI process exits unexpectedly. For your launchd-scheduled agents, you need a Python-level wrapper:

```

import asyncio
import time
from claude_agent_sdk import query, ClaudeAgentOptions, ResultMessage

async def run_with_retry(prompt: str, options: ClaudeAgentOptions, max_retries: int):
    """Retry the entire agent run with exponential backoff."""
    for attempt in range(max_retries):
        try:
            result = None
            async for message in query(prompt=prompt, options=options):
                if isinstance(message, ResultMessage):
                    result = message
                if message.is_error:
                    raise RuntimeError(f"Agent error: {message.subtype}")
            return result
        except (ConnectionError, ProcessError) as e:
            if attempt == max_retries - 1:
                raise
            delay = 2 ** attempt * 5 # 5s, 10s, 20s
            logger.warning(f"Attempt {attempt+1} failed: {e}. Retrying in {delay}s")
            await asyncio.sleep(delay)
    except RuntimeError:

```

```
# Agent-level errors (bad output, context issues) — don't retry
raise
```

Key error types to catch:[1]

- `CLINotFoundError` — Claude Code CLI not installed (fatal, don't retry)
- `CLIConnectionError` — Connection to CLI failed (transient, retry)
- `ProcessError` — CLI process failed with `exit_code` and `stderr` (inspect before retrying)
- `CLIJSONDecodeError` — Malformed output from CLI (likely transient)

Graceful Degradation for Tool Failures

When your `vault_inject` MCP tool fails, the SDK doesn't crash—Claude sees the error response and can adapt. You can make this more explicit through your skill prompt:

Error Handling (in SKILL.md)

If `vault_inject` fails:

1. Fall back to direct Write tool to the same file path
2. If Write also fails, log the intended content to stdout
3. Never silently drop content — always record what was intended

This is the "prompt engineering as error handling" pattern. Since Claude handles tool errors in-context, your reliability strategy is partly a prompt design problem.[2]

ClaudeAgentOptions Configuration for Autonomous Agents

The Full Recommended Configuration

Based on the official SDK reference and your existing setup:[3][1]

```
from claude_agent_sdk import ClaudeAgentOptions
from lib.custom_tools import create_vault_mcp_server
```

```
options = ClaudeAgentOptions(  
    # System prompt: use Claude Code preset + your skill prompts appended  
    system_prompt={  
        "type": "preset",  
        "preset": "claude_code",  
        "append": skill_prompt_text, # Loaded via your skill_loader.py  
    },  
  
    # Permission & safety  
    permission_mode="acceptEdits",  
    setting_sources=["project"], # Inherits hooks from .claude/settings.json  
    max_turns=30,  
    max_budget_usd=0.50,  
  
    # Model configuration  
    model="claude-sonnet-4-20250514",  
    fallback_model="claude-haiku-4-5-20251001",  
  
    # Tool configuration  
    allowed_tools=[  
        "Read", "Write", "Edit", "Glob", "Grep",  
        "mcp__vault-tools__vault_inject",  
    ],  
    disallowed_tools=["Bash"], # Explicit deny for safety  
  
    # MCP servers  
    mcp_servers={"vault-tools": create_vault_mcp_server()},  
  
    # Working directory  
    cwd="/Users/seanwinslow/Code-Brain/claude-code-superuser-pack",  
  
    # Context management  
    max_thinking_tokens=8000, # Reduces context pressure  
)
```

acceptEdits vs dangerouslySkipPermissions

This is the central tradeoff for scheduled autonomous agents:

Dimension	acceptEdits	bypassPermissions (aka dangerouslySkipPermissions)
File edits (Read/Write/Edit)	Auto-approved	Auto-approved
Bash commands	Still requires approval (blocks)	Auto-approved
MCP tool calls	Auto-approved	Auto-approved
Network access	Controlled by hooks	Unrestricted unless hooked
Risk level	Moderate — no arbitrary code execution	High — full system access
Recommended for	Vault-writing agents, file I/O tasks	CI/CD pipelines, sandboxed containers only

Recommendation for your setup: Stay with acceptEdits. Your Daily Driver only needs file I/O and MCP tools — it doesn't need Bash. By keeping Bash out of allowed_tools, you get the safety of acceptEdits without ever hitting a permission prompt that would block the autonomous run.[4][5][3]

There is a documented issue where --allowedTools may be ignored with bypassPermissions mode — use --disallowedTools instead, which works correctly in all modes. In the SDK, this maps to the disallowed_tools parameter.[4]

If you later need Bash for specific commands (e.g., gh CLI for GitHub integration), use the can_use_tool callback for fine-grained control instead of switching to bypass mode:[1]

```

async def selective_bash_permission(tool_name, input_data, context):
    if tool_name == "Bash":
        command = input_data.get("command", "")
        # Allow only specific safe commands
        safe_prefixes = ["gh pr list", "gh issue list", "git log", "git status"]
        if any(command.startswith(p) for p in safe_prefixes):
            return PermissionResultAllow(updated_input=input_data)
        return PermissionResultDeny(
            message=f"Bash command not in allowlist: {command}",
            interrupt=False # Don't kill the run, just deny this tool
        )
    return PermissionResultAllow(updated_input=input_data)

options = ClaudeAgentOptions(
    can_use_tool=selective_bash_permission,
    allowed_tools=["Read", "Write", "Edit", "Glob", "Grep", "Bash",
                  "mcp__vault-tools__vault_inject"],
    permission_mode="acceptEdits",
)

```

query() vs ClaudeSDKClient

For your scheduled autonomous agents, `query()` is the right choice. It creates a new session each time, which is exactly what launchd-triggered runs need. `ClaudeSDKClient` is for interactive multi-turn conversations where you need to maintain state across exchanges and support interrupts.[1]

Feature	<code>query()</code>	<code>ClaudeSDKClient</code>
Session	New each call	Persistent
Custom tools	Not supported	Supported
Hooks (SDK-level)	Not supported	Supported
Use case	Scheduled batch tasks	Interactive apps

Important caveat: The `query()` function does **not** support SDK-level hooks or custom `@tool` definitions directly. However, your setup already handles this correctly — you're using `setting_sources=["project"]` to inherit file-system hooks (like `block-secrets.py`) from `.claude/settings.json`, and your custom `vault_inject` tool is delivered via MCP server rather than the `@tool` decorator. This is the right pattern for `query().[3][1]`

If you later need SDK-level programmatic hooks (Python callbacks rather than shell commands), you'll need to switch to `ClaudeSDKClient`.

MCP Server Architecture

In-Process vs External MCP Servers

Your current approach — an in-process MCP server created with `create_sdk_mcp_server()` — is the simpler and more reliable option for a single-tool setup like `vault_inject.[3][1]`

Approach	Pros	Cons
In-process (<code>McpSdkServerConfig</code>)	No separate process to manage; no IPC failure modes; tools defined in Python with <code>@tool</code> decorator	Runs in the same process; if it crashes, the agent crashes
External stdio (<code>McpStdioServerConfig</code>)	Process isolation; can be shared across agents; independent restart	Extra process management; startup latency; IPC can fail silently
External HTTP/SSE (<code>McpHttpServerConfig</code>)	Network-accessible; can run on remote hosts; language-agnostic	Network failure modes; auth complexity; latency

Recommendation: Keep `vault_inject` in-process. It's a simple file I/O operation — the overhead of process isolation isn't worth the

complexity. Switch to external servers only for tools that:

- Need to run as long-lived daemons (e.g., Jira MCP, Obsidian MCP server)
- Are shared across multiple concurrent agent instances
- Have their own dependency trees that conflict with your agent's environment

For your planned Jira integration, use the external stdio pattern:[3]

```
mcp_servers = {
  "vault-tools": create_vault_mcp_server(),      # In-process (simple)
  "jira": {                                       # External stdio
    "type": "stdio",
    "command": "mcp-atlassian",
    "args": ["--jira-url", "https://your-instance.atlassian.net"],
    "env": {"JIRA_API_TOKEN": os.environ["JIRA_API_TOKEN"]},
  },
}
```

MCP Failure Modes to Watch

1. **Stdio server startup failure:** The external process fails to launch. The SDK will report a `ProcessError`. Solution: validate the MCP server binary exists before calling `query()`.
 2. **Tool timeout:** MCP tool calls can hang if the backing service is unresponsive. The SDK doesn't expose per-tool timeouts directly — use `max_turns` as a circuit breaker.
 3. **Schema mismatch:** If your `@tool` decorator's `input_schema` doesn't match what Claude sends, you get a silent failure (tool returns an error to Claude, who adapts). Add validation inside the tool handler.
 4. **Token cost of tool descriptions:** Each MCP tool consumes context tokens for its description. At scale (10+ tools), this meaningfully reduces available context. For agents with many tools, consider Cloudflare's "Code Mode" pattern where tool discovery is progressive rather than upfront.[6][7]
-

Advanced Cost Control

Beyond Simple Budget Caps

Your `max_budget_usd=0.50` cap is the baseline. Here are patterns for more sophisticated control:[3]

1. Early Termination via Turn Monitoring

The `ResultMessage` gives you `total_cost_usd`, `num_turns`, and `duration_ms`. But you can also monitor **during** the run by tracking `AssistantMessage` instances:[1]

```
async def run_with_cost_guard(prompt, options, cost_warn=0.30, turn_war
    turns = 0
    last_text = ""

    async for message in query(prompt=prompt, options=options):
        if isinstance(message, AssistantMessage):
            turns += 1
            for block in message.content:
                if isinstance(block, TextBlock):
                    last_text = block.text

        if isinstance(message, ResultMessage):
            record_run(
                cost_usd=message.total_cost_usd,
                turns=message.num_turns,
                duration_ms=message.duration_ms,
            )
    return message
```

Note: You cannot interrupt a `query()` call mid-stream — only `ClaudeSDKClient` supports `interrupt()`. For `query()`, the `max_turns` and `max_budget_usd` caps are your only mid-run circuit breakers.[1]

2. Loop Detection via PostToolUse Hooks

Agents sometimes loop — reading the same file repeatedly, or writing and overwriting the same section. Detect this with a hook:

```
# In .claude/settings.json (inherited via setting_sources=["project"])
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "python3 .claude/hooks/detect_loops.py"
      }]
    }]
  }
}
```

```
# .claude/hooks/detect_loops.py
import json, sys, os
from collections import Counter

# Read tool use history from a temp file
history_file = "/tmp/agent_tool_history.jsonl"
input_data = json.loads(sys.stdin.read())

tool_name = input_data.get("tool_name", "")
tool_input = json.dumps(input_data.get("tool_input", {}), sort_keys=True)
entry = f"{tool_name}:{tool_input}"

# Append to history
with open(history_file, "a") as f:
    f.write(entry + "\n")

# Check for loops (same exact call 3+ times)
with open(history_file) as f:
    calls = [line.strip() for line in f]
```

```

counter = Counter(calls[-10:]) # Last 10 calls
for call, count in counter.items():
    if count >= 3:
        # Signal to Claude that it's looping
        result = {
            "systemMessage": f"WARNING: You have made the identical tool call '
        }
        print(json.dumps(result))
        sys.exit(0)

print(json.dumps({}))

```

3. Per-Model Cost Optimization

Use claude-haiku-4-5 for simpler agents (inbox processing, routine reviews) and claude-sonnet-4 for complex reasoning (weekly reviews, project analysis):[2]

```

# config.toml
[agents.daily_driver]
model = "claude-sonnet-4-20250514"
max_budget_usd = 0.50

[agents.process_inbox]
model = "claude-haiku-4-5-20251001"
max_budget_usd = 0.10 # Haiku is ~10x cheaper

```

4. Cost Tracking via ResultMessage

The authoritative cost comes from ResultMessage.total_cost_usd. Token usage attribution is on AssistantMessage instances, and multiple tool uses in one turn share the same message.id — only charge once per ID to avoid double-counting.[8][9]

```

if isinstance(message, ResultMessage):
    # This is the definitive cost for the run
    cost = message.total_cost_usd
    turns = message.num_turns

```

```
duration = message.duration_ms

# Record to your CSV
record_run(
    log_dir=config.log_dir,
    agent_name="daily_driver",
    mode=mode,
    status="success" if not message.is_error else "error",
    cost_usd=cost,
    duration_ms=duration,
    turns=turns,
)
```

Context Window Management for Multi-Step Tasks

The Core Problem

Claude Code uses a ~200K token context window. Automatic compaction triggers at approximately 80% usage (~160K tokens), summarizing old messages to free 40–60% of context. For your Daily Driver's morning mode (which reads yesterday's note, project notes, and creates today's note), this is unlikely to be an issue within 30 turns. But for the weekly review agent reading 7 daily notes, context pressure is real.[10]

Anthropic's Recommended Pattern for Long-Running Agents

Anthropic's engineering team published a specific solution: use an **initializer agent** for the first context window and a **coding agent** for incremental progress, connected by a progress file and git history.[11]

Key failure modes they identified and solutions:[11]

Problem	Solution
Agent tries to do too much at once	Prompt for incremental progress — one feature/task per session
Agent declares victory prematurely	Maintain a structured task list (JSON, not Markdown — less likely to be inappropriately edited)
Agent leaves environment in broken state	Commit progress to git with descriptive messages; write progress summaries
Next session has no memory	Progress file + git log provide quick orientation

For your setup, the analog is: your `claude-progress.txt` equivalent is the daily note itself plus `agent-run-history.csv`. The morning agent reads yesterday's note to orient, and the evening agent reads today's note to review.

PreCompact Hook for Preserving Critical Context

The PreCompact hook fires before compaction occurs. Use it to ensure critical context survives:[12][10]

```
# SDK-level hook (requires ClaudeSDKClient, not query())
async def preserve_context_before_compaction(input_data, tool_use_id, c
    """Save critical state before compaction summarizes it away."""
    trigger = input_data.get("trigger", "auto") # "manual" or "auto"

    return {
        "systemMessage": (
            "BEFORE COMPACTION: Ensure you have written all pending changes
            "After compaction, you will only have a summary of this conversation
            "Verify that your current task's progress is saved to the daily note."
        )
    }
}
```

For file-system hooks (compatible with `query()`):[13]

```
{
  "hooks": {
    "PreCompact": [{
      "hooks": [{
        "type": "command",
        "command": "echo 'Compaction triggered' >> vault/90_system/agent-..."
      }]
    }]
  }
}
```

The 1M Token Beta

The SDK exposes a `betas` parameter with `"context-1m-2025-08-07"` as an available option. This dramatically extends the context window and may eliminate compaction issues for your weekly review agent: [1]

```
options = ClaudeAgentOptions(
  betas=["context-1m-2025-08-07"],
  max_thinking_tokens=16000, # More room for reasoning with larger conte
  # ... other options
)
```

Tradeoff: Larger context = higher token costs per turn. Monitor costs closely if you enable this.

Ecosystem: Community Tools and Middleware

Observability

disler/claude-code-hooks-multi-agent-observability (556 stars)[14]

The most mature community observability solution. Architecture: Claude agents → hook scripts → HTTP POST → Bun server → SQLite → WebSocket → Vue client. Provides real-time monitoring of multiple concurrent agents with session tracking, event filtering, and live

visualization including a pulse chart. Captures all hook event types: PreToolUse, PostToolUse, Stop, SubagentStop, PreCompact, SessionStart/End, and UserPromptSubmit. Straightforward to integrate — copy the .claude directory to your project and point the hook commands at the server.

nexus-labs-automation/agent-observability[15]

A Claude Code plugin with 14 specialized skills for observability. Includes /instrument and /audit slash commands that generate tiered instrumentation plans (T0–T4 from foundation tracing to multi-agent coordination). Supports integration with Langfuse, LangSmith, Arize, OpenTelemetry, Sentry, and Datadog. Has 9 framework-specific guides including one for the Claude Agent SDK.

Orchestration

FastHarness[2]

Created by Prassanna Ravishankar to extract repeated boilerplate from Claude Agent SDK agents. Provides an @harness.agentloop() decorator that handles FastAPI/A2A protocol wiring, async message iteration, message ID deduplication for cost tracking, step logging, and structured output extraction. Reduced a 141-line agent to 70 lines. Includes CostTracker with configurable warn/error thresholds and ConsoleStepLogger.

```
from fastharness import FastHarness, CostTracker, ConsoleStepLogger
from fastharness.client import HarnessClient
from fastharness.core.context import AgentContext

harness = FastHarness(name="daily-driver", version="0.1.0")
cost_tracker = CostTracker(warn_threshold_usd=0.30, error_threshold_usd=C

@harness.agentloop(
    name="morning-planning",
    system_prompt=SYSTEM_PROMPT,
    model="claude-sonnet-4-20250514",
    output_format={"type": "json_schema", "schema": PlanSchema.model
)
async def morning(prompt: str, ctx: AgentContext, client: HarnessClient):
```

```
client.telemetry_callbacks.append(cost_tracker)
return await client.run(prompt)
```

OmoiOS[16]

An open-source orchestration layer that generates task DAGs from specifications, runs tasks in isolated Daytona sandboxes, validates against acceptance criteria, and auto-retries with failure context. Uses `structured_output()` with Pydantic models for typed, validated data from every LLM call. Stack: FastAPI, SQLAlchemy, PostgreSQL, pgvector, Redis pub/sub.

Multi-Agent Frameworks

Microsoft Agent Framework + Claude Agent SDK[17]

Official integration (released January 2026) that wraps Claude agents in Agent Framework's BaseAgent interface. Enables composition with Azure OpenAI, GitHub Copilot, and other agents in sequential, concurrent, handoff, and group chat workflows. Useful if you later want your Daily Driver to coordinate with agents running on different models.

avivl/claude-007-agents[18]

A system of specialized agents across 14 categories with a coordination layer. Features organizational memory via Basic Memory MCP. Includes exponential planning, parallel coordination, and session management agents.

What's Missing from the Ecosystem

As of February 2026, there are notable gaps:

- **No official retry/circuit-breaker middleware** — you build this yourself (pattern above)
- **No shared standard for agent run logs** — your CSV approach is as good as anything
- **No production-grade testing framework** — the SDK has typed errors (`ClaudeSDKError`, `ProcessError`) but no mock/stub utilities for testing agent behavior without API calls. Your `--dry-run` pattern is the community standard[3]

- **No built-in loop detection** — must be implemented via hooks (pattern above)
-

Version-Specific Notes and Recent Changes

- The package is `claude-agent-sdk` (renamed from `claude-code-sdk`). Import as `from claude_agent_sdk import query, ClaudeAgentOptions`.^[19]
 - `total_cost_usd` is a top-level field on `ResultMessage`, **not** nested under `usage`. A documentation bug existed as of January 2026 showing incorrect access via `result.usage.total_cost_usd`.^[8]
 - The `betas` field supports `"context-1m-2025-08-07"` for 1M token context.^[1]
 - `setting_sources` defaults to `None` (no filesystem settings loaded). You must explicitly include `"project"` to load `CLAUDE.md` files and `.claude/settings.json` hooks.^[1]
 - `enable_file_checkpointing=True` enables `rewind_files()` on `ClaudeSDKClient` for restoring file state to a specific point — useful for recovery from bad edits.^[1]
 - Python SDK's `query()` does not support hooks or custom tools directly. Use `ClaudeSDKClient` for those, or rely on filesystem hooks via `setting_sources=["project"]`.^[1]
-

References

1. [Agent SDK reference - Python - Claude API Docs](#) - The Python SDK provides two ways to interact with Claude Code: REPLs
Response-driven logic. Restore ...
2. [Agent Frameworks Ain't Agent Building | Prassanna Ravishankar](#) - Claude Code is already a multi-step agent runtime. It handles tool sequencing, error recovery, conte...
3. [agents-sdk.md](#) - # Agents SDK Integration Guide

How the Claude Agent SDK extends the Superuser Pack from an intera...

4. [Claude Code Autonomous Mode: Complete Guide to](#) - In this comprehensive guide, we will explore how to configure Claude Code in autonomous mode, using ...

5. [claude --dangerously-skip-permissions - PromptLayer Blog](#) - The -dangerously-skip-permissions flag lets Claude Code run end-to-end without approvals. This guid...
6. [MCP Server Observability: Monitoring, Testing & Performance Metrics](#) - Here's what MCP server crashes really cost—and the three-layer monitoring framework that catches fai...
7. [Code Mode: give agents an entire API in 1,000 tokens](#) - Want an agent that manages DNS? Add the DNS MCP server. Want Workers logs? Add the Workers Observabi...
8. [\[DOCS\] Incorrect property access for total_cost_usd in cost-tracking ...](#) - The property access result.usage.total_cost_usd is incorrect based on the SDK's own type definitions...
9. [Claude Agent SDK - Deepak Karkala](#) - Total Cost: The final Result message contains the authoritative total_cost_usd for the session. 7. C...
10. [Context Management - SFEIR Institute](#) - Context management in Claude Code determines the quality and speed of every interaction with the age...
11. [Effective harnesses for long-running agents - Anthropic](#) - We developed a two-fold solution to enable the Claude Agent SDK to work effectively across many cont...
12. [Intercept and control agent behavior with hooks - Claude API Docs](#) - Hooks let you intercept agent execution at key points to add validation, logging, security controls,...
13. [Automate workflows with hooks - Claude Code Docs](#) - Re-inject context after compaction. When Claude's context window fills up, compaction summarizes the...
14. [GitHub - disler/claude-code-hooks-multi-agent-observability](#) - This system provides complete observability into Claude Code agent behavior by capturing, storing, a...
15. [Claude Code plugin for AI agent observability - GitHub](#) - This Claude Code plugin provides best practices, hooks against anti-patterns, and ready-to-use templ...
16. [I built an orchestration layer on Claude Agent SDK ... - Reddit](#) - I built an orchestration layer on Claude Agent SDK — agents execute from specs in sandboxes and vali...
17. [Build AI Agents with Claude Agent SDK and Microsoft Agent ...](#) - Microsoft Agent Framework now integrates with the Claude Agent SDK, enabling you to build AI agents ...

18. [avivl/claude-007-agents: A unified AI agent orchestration system ...](#) - A unified AI agent orchestration system featuring 10's of specialized agents across 14 categories fo...
19. [Anthropic Claude SDK with MCP: enterprise deployment guide for AI ...](#) - Complete Observability: Monitor which tools Claude accesses, what operations are performed, and trac...